

SWASTHAM.AI · PROOF OF CONCEPT · PROJECT REPORT

Multilingual Health RAG

One verified answer for every language, by text or by voice

English · हिन्दी · Hinglish

Prepared by	Chetanya Jain
Prepared for	Founder, Swastham.ai
Date	28 September 2026
Work period	23 – 28 September 2026
Status	Approved proof of concept, deployed and running

Live demo	https://swastham-rag-demo.vercel.app
Source code	https://github.com/chetanya-swastham/RAG_demo_swastham.ai
Tech reference	<code>README.md</code> and <code>IMPLEMENTATION.md</code> in the repository

1 Executive summary

Swastham users ask health questions in English, Hindi and Hinglish. When an AI model answers each language separately, the answers **drift apart**: facts, numbers and even safety warnings differ between languages. For a health product, that is a patient-safety risk.

I designed and built a retrieval-augmented generation (RAG) system that removes this risk by design. **Every question, in any language, is answered once in English from the verified knowledge base.** Hindi and Hinglish users receive a **checked translation** of that exact answer. The translation is automatically back-translated and must keep every number, safety caution and recommendation. If it doesn't, the answer is blocked rather than shown or spoken.

3

languages from one answer
path: English, Hindi, Hinglish

81 / 81

automated unit tests passing

14 / 14

Hinglish questions retrieve the
same evidence as English

7 / 7

parity checks passed on the
insulin, BMI, pregnancy and
hunger samples

What was delivered

- **The brief, fully met:** an English knowledge base indexed in FAISS, a structured Answer Plan, one canonical English answer, lightweight validation, and a UI to compare English and Hindi side by side.
- **Beyond the brief**, each built on the same "one answer" rule:
 - fail-closed translation safety gates;
 - question canonicalisation with an answer store (same question, same answer, every time);
 - full **Hinglish** support;
 - a trilingual evaluation suite;
 - a **voice assistant** and a hands-free **voice agent** that understands follow-up questions.
- **Deployed to the cloud** as a container on Vercel, and **documented** for handover (README, implementation guide, 81 offline tests).

How problems were solved

Early testing showed Hindi answers quietly losing a doctor referral and softening an insulin warning. I built a runtime trace that logs every retrieval and model call, found the root cause (the translation step, not the architecture), and fixed it in the prompts, the medical glossary and new safety gates. **I never lowered a quality threshold to make a test pass.** Section 6 walks through these investigations.

Try it: swastham-rag-demo.vercel.app. Ask "मैं टाइप 2 डायबिटीज के लिए इंसुलिन लेता हूँ। क्या मैं दीक्षित की दिन में दो बार भोजन वाली योजना अपना सकता हूँ?" on the Ask tab and open the *Pipeline trace* to see every step and check. Appendix A gives the 5-minute demo script and setup steps.

2 The problem

A common way to support Hindi is to let the AI model answer a Hindi question directly in Hindi. That produces **two independent answers** to the same question, one per language. They are generated separately, so they can say different things:

- **Different facts or detail:** the Hindi answer is often shorter and less specific.
- **Lost numbers:** a BMI cut-off or a walking target can change or disappear.
- **Lost safety cautions:** for example, "people on insulin must adjust their dose with their doctor" can go missing. This is the most serious risk.

Why it matters for Swastham: a Hindi-speaking user with type 2 diabetes who is on insulin must get exactly the same safety warning as an English-speaking user. An answer that is "mostly the same" is not acceptable in healthcare.

The original brief

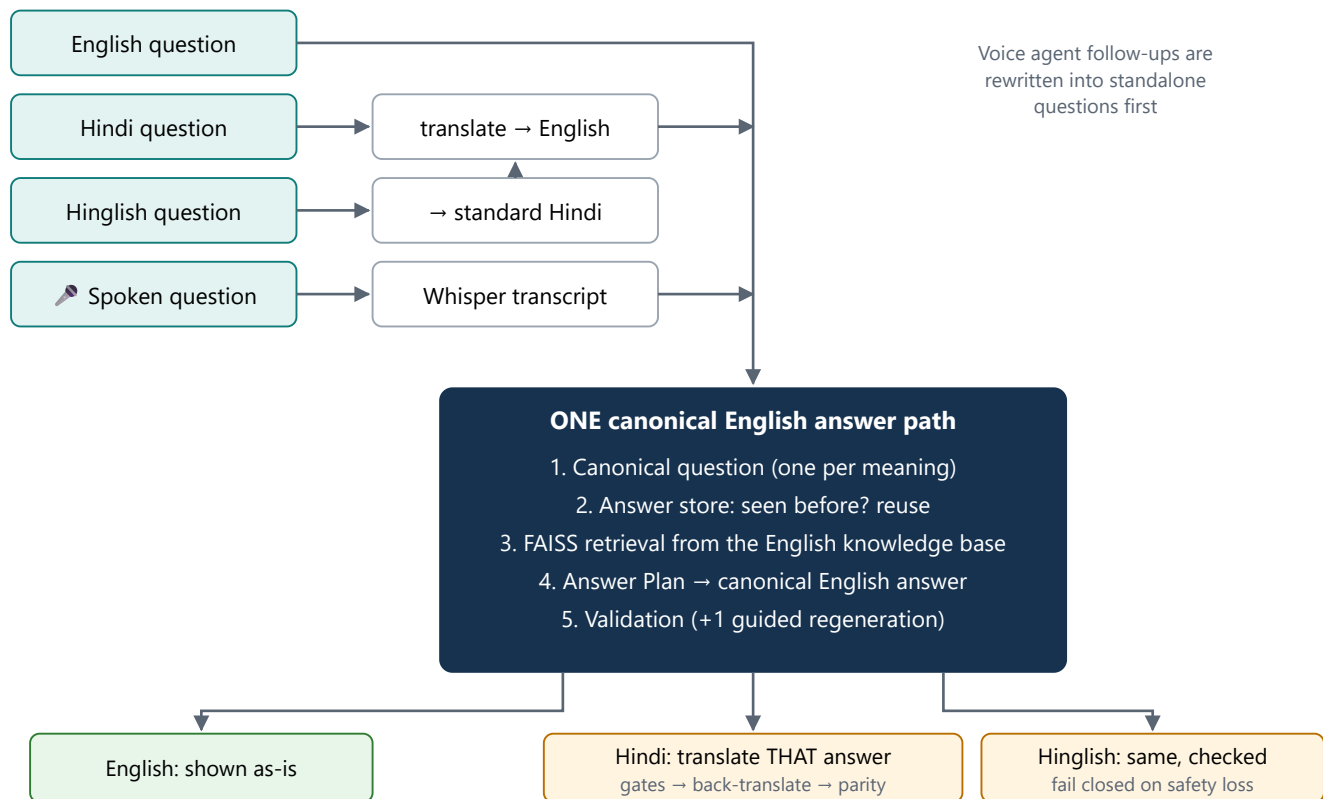
Take an English knowledge document on obesity, diabetes and the Dixit Lifestyle, and make sure the same question in English and Hindi rests on **the same evidence and the same underlying answer**. Hindi must never reason on its own. The brief also asked to keep the stack simple: Python, PyMuPDF, sentence-transformers, FAISS and one LLM, with no separate Hindi vector database, no agent frameworks, no hybrid search and no reranking.

Brief vs. delivered

Brief asked for	Delivered
Ingestion: extract → chunk → English embeddings → FAISS	Section-aware chunking with heading context and safety tagging: 83 chunks, 23 of them marked safety-critical. The index is checked against its embedding model
Detect language; translate Hindi to English	Deterministic detection for English, Hindi and Hinglish , with a fixed medical glossary for translation
Answer Plan before generation	A JSON plan (required points, conditions, restrictions, evidence) in which safety cautions become required points
Canonical English answer from evidence only	Yes, plus canonicalisation so that different wordings of one question become the same question, and an answer store so it always gets the same answer
Lightweight validation	Four validation checks with one guided regeneration, plus back-translation parity checks on every Hindi and Hinglish answer, which fail closed
Small UI to compare EN and HI	A three-tab web app: Ask (typed or 🗣️ voice), Parity Compare (EN/HI/Hinglish scorecards against the old baseline) and a hands-free Voice Agent
Keep it simple	Kept to the same simple stack: one English index, one LLM provider, no agent framework. It has also been deployed to the cloud

3 The solution: generate once, translate everywhere

The core design decision is simple and strict: **Hindi and Hinglish never reason on their own**. Every language goes through the same English answer path. Other languages only get a translation, and that translation must pass checks before anyone sees or hears it.



Design principles I applied

Principle	What it means in the system
Generate once, translate everywhere	Only one function writes medical content. Hindi, Hinglish and speech are all derived from that one English answer
Same meaning → same answer	Questions are canonicalised and answers are stored per canonical question, so repeat questions get identical answers in seconds
Validate translations like any model output	Script, number and medical-term gates run on every translation, then a back-translation parity check
Fail closed on safety	A translation that loses a number or a safety caution after one retry is never shown or spoken
Fix causes, not thresholds	Every failure was fixed in the prompt, glossary or pipeline. No threshold was ever lowered
Deterministic and testable	Temperature 0 with a fixed seed. All checks are pure functions, unit-tested offline without an API key

4 How the work progressed

The project ran from 23 to 28 September 2026. Each stage started from a problem found in the previous one.

23 SEP · FOUNDATION

Brief, knowledge base and core pipeline

- Structured the English knowledge base (obesity, diabetes, the Dixit Lifestyle) for retrieval.
- Designed the single answer path: ingestion → FAISS retrieval → Answer Plan → canonical English answer → validation.

24 SEP · MEASURE, THEN FIX

Evaluation set and first round of parity fixes

- Built the index: 83 chunks, 23 tagged safety-critical.
- Wrote a 15-question EN/HI evaluation set, 6 of them safety-critical.
- A runtime trace exposed three translation faults: a Hindi answer echoed in English, a lost obstetrician referral, and an insulin warning whose certainty was weakened.
- Fixed them with glossary entries, prompt rules and new fail-closed translation gates.

24 – 25 SEP · CONSISTENCY

Second round of fixes, canonicalisation and the answer store

- Fixed a tense drift and a glossary over-reach found in the hunger sample.
- Added LLM question canonicalisation, so English and Hindi converge on one question.
- Added a fingerprinted answer store that resets itself when prompts, the model or the knowledge base change.
- Parity is now checked on every Hindi answer, not only on the Compare tab.
- Built the EN/HI evaluation runner and the parity trace tool, and delivered the first presentation.

26 SEP · NEW LANGUAGE

Hinglish support and trilingual evaluation

- Hinglish is common with Indian users but defeats plain embeddings (only 0.21 evidence overlap).
- Built a deterministic Hinglish detector, a spelling normaliser, protected medical terms and a checked Hinglish-to-Hindi conversion.
- Built a trilingual evaluation suite with a 14-question dataset, showing evidence overlap of 1.00 with English.

28 SEP · VOICE, DOCUMENTATION AND DEPLOYMENT

Voice assistant, voice agent, documentation and cloud deployment

- Added speech in and out around the same verified pipeline: Whisper speech-to-text and neural text-to-speech.
- Built a hands-free voice agent with pause detection, 3-turn memory for follow-ups and barge-in.
- Tested live in Hindi and Hinglish.
- Wrote the README and implementation guide, bringing the suite to 81 unit tests.
- Packaged the app as a container and deployed it to Vercel.

5 Features I added

Each feature started from a concrete need. Each one keeps the core rule: one canonical English answer per question, with every other language a checked translation of it.

SAFETY Parity checks and fail-closed translation gates

Why	A translation can silently drop a warning. The first traces proved it happens.
How	Every translation passes these checks: • Gates: correct script, identical numbers, medical terms present. • Back-translation to English, then five parity checks: evidence overlap, numbers, safety cautions, recommendations and content coverage. • On failure, one retry with the failed checks as feedback.
Outcome	If the numbers or safety cautions are still wrong after the retry, the answer is blocked . A user never sees or hears an unverified translation.

CONSISTENCY Question canonicalisation and answer store

Why	"Can I eat fruit?" and "Are fruits OK for me?" are the same question and must get the same answer.
How	Each question is rewritten into one standard English question, cached per wording. Validated answers and passing translations are stored per canonical question. A fingerprint of the model, prompts, glossaries and knowledge base resets the store automatically when any of them change.
Outcome	Repeat questions are answered identically in a few seconds, instead of the 50–80 seconds a new question takes on the free API tier.

LANGUAGE Hinglish (Romanised Hindi) input and output

Why	Many users type Hindi in English letters, mixed with English words. Embedding that text directly retrieves the wrong evidence (overlap 0.21 with English).
How	Hinglish questions go through a pipeline and join the existing Hindi path. Answers come back through the same checks as Hindi. • Detection: scored strong and weak marker words, with no model call. • Normalisation: one standard spelling per Hindi word (<i>mai</i> → <i>main</i>, <i>hu</i> → <i>hoon</i>). • Protection: medical terms such as <i>HbA1c</i>, <i>glimepiride</i> and <i>insulin</i> are masked before conversion. • Conversion: a checked step from Roman script to Devanagari.
Outcome	Evidence overlap with English rose from 0.21 to 1.00 , on 14 of 14 questions. All four forms below reach the same canonical question and the same evidence:

```
EN Can I eat fruits if I have diabetes?
HI अगर मुझे डायबिटीज है तो क्या मैं फल खा सकता हूँ?
HG Mujhe diabetes hai, kya main fruits kha sakta hu?
HG kya mai fruit kha sakta hu diabetes me
```

QUALITY Trilingual evaluation suite

Why	"It looks right" is not evidence. Consistency across languages had to be measured.
How	A dataset of 14 EN/HI/Hinglish triples (5 safety-critical) with spelling variants. The suite measures: • retrieval consistency, evidence overlap and missing evidence; • plan consistency and answer similarity; • hallucination rate and parity. Each language runs <i>separately</i>, so convergence is measured rather than forced. The Hinglish normaliser is compared against two baselines. Reports are produced as Markdown, JSON and CSV.

VOICE Voice assistant and hands-free voice agent

Why	Many users prefer speaking to typing, especially in Hindi.
How	Voice only adds input and output around the same verified pipeline, so what is spoken is the checked answer. • Speech in: Whisper, with Urdu-script output corrected to Hindi and silence rejected. • Speech out: neural Hindi and Indian-English voices. • Voice agent: microphone calibration to room noise, pause detection and 3-turn memory. It rewrites follow-ups such as "और अगर मैं प्रेग्नेंट हूँ तो?" into standalone questions, lets you interrupt it (barge-in), and shows a ✓ verified badge.
Outcome	The live Hindi round trip was transcribed exactly. A two-turn pregnancy follow-up was rewritten correctly, answered in Hinglish, and passed validation and parity. If it cannot verify an answer, the agent says so and refers the user to their doctor.

PRODUCT Parity Compare UI, web app and cloud deployment

What	A three-tab web app with no build step. • The Ask tab shows a full pipeline trace and the evidence behind each answer. • The Parity Compare tab scores English, Hindi and Hinglish side by side against the old "answer directly in Hindi" baseline. It refuses to compare different questions (<i>Question Intent Mismatch</i>). The app is containerised with CPU-only PyTorch and the embedding model preloaded, and deployed on Vercel with the API key kept in the platform's environment variables.
------	---

6 Problem solving: case studies

I found these problems with a **runtime trace tool** (`trace_parity.py`) that logs every retrieval and every model call for the sample questions. The trace showed that the architecture was right, with one retrieval, one generation, and the exact answer passed to the translator. **The translation itself was unfaithful.** So every fix went into the translation step.

CASE 1 A check that passed and hid a bug (BMI)

Symptom	The BMI sample "passed" parity.
Root cause	The translator had returned the English text unchanged , so English was being compared with English.
Fix	A gate that rejects a "Hindi" answer that is not mostly Devanagari, plus a rule that the output must always be Hindi, with the instruction repeated after the source text.
Lesson	Every check needs a sanity check on its inputs. A passing check can hide a bug.

CASE 2 Doctor referral lost in translation (pregnancy)

Symptom	The safety check failed on the Hindi pregnancy answer.
Root cause	"Obstetrician" was translated as प्रसूति विशेषज्ञ ("obstetric specialist"), which did not back-translate as a doctor referral.
Fix	Referral roles (clinician, obstetrician, paediatrician, endocrinologist, healthcare team) were added to both glossaries, with a rule to keep the specific professional named.

CASE 3 An insulin warning made weaker (insulin)

Symptom	The recommendation check failed.
Root cause	"Clinician" collapsed to "doctor", and "will almost certainly need to be adjusted" became "adjusted fairly precisely". The certainty of a safety instruction was weakened.
Fix	Prompt rules to keep certainty words and the named professional, and the answer sent inside explicit delimiters. Exact-number gates with one retranslation, then fail closed.

CASE 4 Tense drift and a glossary over-reach (hunger)

Symptom	The recommendation check failed on the hunger sample.
Root cause	Three faults: • "Take medicines" became "are taking", losing the instruction. • "Diabetes" was expanded to "type 2 diabetes" from the glossary. • A heading was left in English.
Fix	Separate glossary entries for related terms, a rule to use a glossary term only where the English uses that exact term , and rules to keep tense and translate headings.
Result	Insulin, BMI, pregnancy and hunger all pass 7 of 7 checks, with no threshold or scoring logic changed .

Other issues solved along the way:

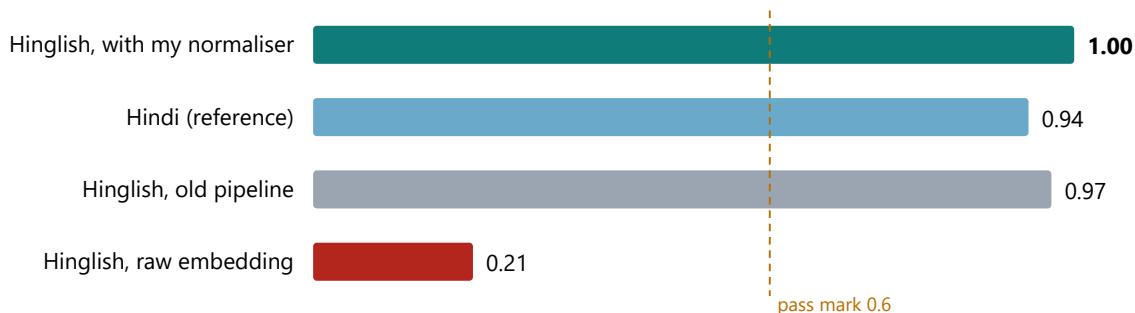
- Whisper labels spoken Hindi as Urdu, so the app re-transcribes it as Hindi.
- Whisper invents "Thank you." on silence, so silent segments are filtered out.
- The model's reasoning tokens were cutting answers short, so the output budget was raised and cut-offs now raise an error instead of showing a partial answer.

- Keyword checks matched the middle of words (*kid* in *kidney*), so they now match whole words only.
- Stale stored answers are now discarded automatically by the fingerprint.

7 Results and quality evidence

Hinglish retrieval: does the question reach the same evidence as English?

14 questions, retrieval-only evaluation, 26 Sep 2026 ([reports/trilingual_report.md](#)). Evidence overlap is the Jaccard overlap of retrieved chunks with the English question's chunks. The pass mark is 0.6.



Retrieval of the Hinglish question	Mean overlap	Passed	Missing evidence	Same canonical question
With normaliser	1.00	14/14	0	10/14
Old pipeline (Hinglish treated as English)	0.97	14/14	1	7/14
Raw embedding	0.21	4/14	42	0/14
Hindi, for reference	0.94	14/14	2	8/14

Hinglish was detected in 14 of 14 questions, and 0 of 14 English questions were wrongly detected as Hinglish.

Full answer evaluation (samples T1, T2, T9)

Language	Parity passed	Mean answer similarity to English	Mean plan consistency
Hindi	3/3	0.93	0.89
Hinglish	3/3	0.97	0.92

Other quality evidence

Area	Result
Automated tests	81 / 81 pass (re-run for this report on 28 Sep 2026). 32 parity, 28 Hinglish and 21 voice tests. They run offline, with no API key, microphone or network
Parity trace	Insulin, BMI, pregnancy and hunger samples: 7/7 checks each
Voice (live)	The Hindi speech round trip was transcribed exactly: "मुझे हर समय भूख क्यों लगती है?". The follow-up was correctly rewritten and passed parity in Hinglish
Deployment	Live app is responding. Its <code>/api/status</code> reports the model key and voice both enabled (checked on 28 Sep 2026)

Engineering practices

- **Determinism:** temperature 0 and seed 42 on every model call, with stored answers for repeatable output.
- **No silent fallback:** truncated or empty model output raises an error, so the demo never shows a partial or invented answer.
- **Resilience:** automatic retries on rate limits, server errors and timeouts (up to 8 attempts).
- **Safe storage:** the answer store is written atomically under a lock, and only validated answers are stored.

- **Separation of concerns:** ingestion, LLM layer, pipeline, validator, Hinglish, voice, server and UI are separate modules. Adding a new language touches only the translation layer (see the implementation guide, §17).

8 Limitations and recommended next steps

This is a proof of concept. These are its known limits, stated plainly, with the step I recommend for each before production.

Limitation today	Recommended next step
The checks are keyword-based, so they raise some false alarms. For example, "talk with your doctor" can be missed where the check looks for "consult a healthcare professional"	Add meaning-level checks (an entailment model or an LLM judge) alongside the keyword checks
A new question takes 50–80 s on the free API tier (6–10 model calls). Repeat questions take seconds	A paid tier or a faster model; pre-warm the store with the most common questions
Slightly different canonical wordings ("BMI cut-offs" vs "cut-off") are stored separately	Reuse a stored answer when the evidence is identical and the questions are very similar (cosine $\geq \sim 0.9$)
The medical glossary determines whether advice survives translation	Version the glossary and have a clinician review it. Rerun all samples after each change
The system answers only from the knowledge document	Extend the knowledge base. Re-ingesting resets stored answers automatically
Voice pause detection is energy-based, and Hinglish speech is transcribed in Devanagari	Use a speech-activity model. "Reply in: Hinglish" already handles the reply language

Roadmap to production

1. **Clinical review** of the glossary and the safety-caution rules.
2. **Meaning-level validation** alongside the keyword checks.
3. **More languages** (e.g. Marathi, Tamil). The architecture needs only a new checked translator and a voice for each; retrieval, planning and validation stay unchanged.
4. **Integration** with the Swastham app: authentication, logging and monitoring of parity failures, and a larger evaluation set drawn from real user questions.

What this project demonstrates

Capability	Evidence in this project
RAG system design	An end-to-end pipeline: chunking, embeddings, FAISS, Answer Plan and grounded generation, built within the brief's constraints
AI safety for healthcare	Fail-closed gates, preservation of safety cautions and doctor referrals, and no unverified output
Adding features without breaking the core	Five major features were added on top of one unchanged "one answer" rule
Problem solving	Traced root causes in the runtime and fixed causes, not thresholds (Section 6)
Measurement	Two evaluation suites, baselines and 81 unit tests
Full-stack delivery	Python backend, web UI, browser voice engine, container, cloud deployment and documentation

A Appendix: how to run and demo

Use the live app

Open <https://swastham-rag-demo.vercel.app> in Chrome or Edge. Voice needs microphone permission.

Run it locally

You need Python 3.10+, internet access and a free Groq API key from console.groq.com/keys.

```
git clone https://github.com/chetanya-swastham/RAG_demo_swastham.ai.git
cd RAG_demo_swastham.ai
pip install -r requirements.txt
copy .env.example .env          # then set GROQ_API_KEY in .env
python server.py                # open http://localhost:8000
```

The search index and stored answers are included in the repository, so you don't need to re-ingest. To rebuild the index, run `python ingest.py --source documentRAG.md`. The first run downloads the embedding model (about 90 MB).

Run the tests and evaluations

```
python -m unittest test_parity.py test_hinglish.py test_voice.py # 81 offline tests, no key needed
python evaluate.py                                              # 15 EN/HI pairs vs baseline
python -m evaluation.run_trilingual_eval --retrieval-only        # EN/HI/Hinglish consistency report
```

Suggested 5-minute demo

1. **Ask tab:** ask the Hindi insulin question and open the *Pipeline trace*. It shows the canonical question, the evidence, the Answer Plan, the validation and the parity checks.
2. **Parity Compare:** run the *Insulin (safety)* sample with *Show baseline* ticked. The canonical pipeline passes, and the old approach is scored alongside it.
3. **Voice Agent:** set *Reply in: Hinglish*, then say "I take insulin for type 2 diabetes, can I follow the Dixit two-meal plan?" and follow up with "aur agar main pregnant hoon to?". Interrupt it mid-answer to see barge-in.

Repository map

Path	Purpose
<code>rag_pipeline.py</code>	The single answer path, answer store, Parity Compare and baseline
<code>llm_client.py</code>	Every model call, prompts, medical glossaries, translation gates
<code>validator.py</code>	Answer validation and parity checks (pure functions)
<code>ingest.py</code>	Chunking, embeddings, FAISS index, retrieval
<code>hinglish/</code>	Hinglish detector, spelling normaliser, protected medical terms
<code>voice.py</code> , <code>web/voice.js</code> , <code>web/agent.js</code>	Speech in and out, browser voice engine, voice agent
<code>server.py</code> , <code>web/</code>	Web server, JSON and audio API, user interface
<code>evaluation/</code> , <code>evaluate.py</code> , <code>trace_parity.py</code>	Evaluation suites and the runtime trace tool
<code>README.md</code> , <code>IMPLEMENTATION.md</code>	Setup guide and full technical reference